



SENIOR SOFTWARE ENGINEER INTERVIEW QUESTIONS: DEEP DIVES

Kafka Offsets and Delivery

The senior engineer answer sheet

The payment succeeded. The commit did not. Both of those are true at once.

An offset is not a pointer to what your application did. It is a record of what your consumer group said it had done.

Everything difficult about delivery semantics follows from the gap between those two sentences, and every interview answer that sounds senior is really an answer about where that gap sits in your code.

Every Kafka statement below was checked against Apache Kafka 4.3 documentation, named beside it, on the date at the foot of this sheet.

In scope

- What an offset actually records, and for whom
- At-most-once and at-least-once, both halves of each
- Auto commit, and the window it opens
- Manual commit, sync against async
- Where exactly-once is real and where it is marketing
- Storing the offset with the output
- Isolation level, and the last stable offset

Out of scope

- Broker log segment layout and retention
- Kafka Streams, which manages offsets on your behalf
- Consumer group rebalancing, which is episode 3
- The transactional outbox, which is episode 7

Delivery semantics questions get answered too broadly. Naming the boundary is most of the answer.

01 Worked assumptions

Records per poll	500, the <code>max.poll.records</code> default
Processing time per record	40 ms
Auto commit interval	5 seconds, the default
Records processed in one interval	about 125
Records at risk when the process dies	up to one full interval of work
Payment value at risk	whatever 125 payments are worth to you

These are interview assumptions chosen to make the behaviour concrete. They are not published figures from any provider and no claim is made about any particular deployment.



03 What an offset actually records

An offset is a position in one partition. A committed offset is a position stored for one consumer GROUP, against one partition.

Not per consumer. Not per process. Per group. That is why a partition handed to a different member resumes from a group level fact rather than from anything the previous owner held in memory.

And it records intent to resume, not completion of work. Nothing in Kafka knows whether your side effect happened.

Offsets are stored per partition per consumer group, so a reassignment resumes from the last committed position rather than from wherever the previous owner had reached.

Kafka 4.3, Design: Message Delivery Semantics

The broker has no knowledge of what an application did with a record, so a committed offset can never be evidence that the work completed.

First principles, not vendor documentation

04 The two orderings, and both halves of each

Commit BEFORE processing and you can lose work: the offset says done, the crash says otherwise, and nothing replays it. That is at-most-once.

Commit AFTER processing and you can repeat work: the effect landed, the crash beat the commit, and the record comes back. That is at-least-once, and it is the default.

THE HALF MOST ANSWERS DROP. At-most-once needs the producer to stop retrying as well. A retrying producer can reintroduce exactly the duplication the early commit was meant to prevent, so commit ordering alone does not buy you at-most-once.

Kafka guarantees at-least-once delivery by default, and allows users to implement at-most-once delivery by disabling retries on the producer and committing offsets in the consumer prior to processing a batch of messages.

Kafka 4.3, Design: Message Delivery Semantics

Choosing a delivery semantic is choosing which failure you are willing to have, because a system that never loses and never repeats requires cooperation beyond the broker.

First principles, not vendor documentation

05 Auto commit, and the window it opens

At the time of writing, September 2026, on Kafka 4.3, `enable.auto.commit` defaults to `true` and `auto.commit.interval.ms` defaults to 5000.

That is convenient and it is a five second window in which the offset can move past work that is still in flight, or lag behind work that is already finished. Which direction hurts you depends on what your consumer does.

Auto commit is not wrong. It is a default chosen for the common case of a consumer whose only side effect is inside Kafka. A payment consumer is not that case.

`enable.auto.commit` defaults to `true` on Kafka 4.3.

Kafka 4.3, Consumer Configs

`auto.commit.interval.ms` defaults to 5000 milliseconds on Kafka 4.3.

Kafka 4.3, Consumer Configs

Automatic commits happen on a timer relative to poll calls rather than relative to the completion of application work, so the two can drift apart in either direction.

First principles, not vendor documentation

06 Manual commit, and the sync against async trade

Committing synchronously blocks until the broker acknowledges, and retries on retrieable errors. It is slower and it is certain.

Committing asynchronously does not block, which is faster and means a failed commit is discovered later, if at all. A common shape is `async` during the loop and a final `sync` on shutdown, so speed is bought where a mistake is cheap and certainty is paid for where it is not.

Neither of these makes your processing idempotent. They only narrow the window.

`commitSync` blocks until the commit is acknowledged or fails, while `commitAsync` returns immediately and reports the outcome through a callback.

Kafka 4.3, Consumer Configs

Narrowing the window between the effect and the commit reduces how often a duplicate happens and cannot reduce it to never.

First principles, not vendor documentation



07 Where exactly-once is real, and where it is a slogan

Inside Kafka, reading from a topic, processing, and writing back to a topic, exactly-once is real. A transactional producer commits the output records and the input offsets together, and a `read_committed` consumer only sees committed output.

The moment an effect leaves Kafka, that guarantee stops at the boundary. Kafka says so itself.

So the honest answer in an interview is a question back: exactly once ACROSS WHAT? Name the systems that must agree. That is the sentence that separates a senior answer from a memorised one.

Exactly-once delivery for other destination systems generally requires cooperation with such systems, but Kafka provides the primitives which makes implementing this feasible.

Kafka 4.3, Design: Message Delivery Semantics

Transactional producers with read-committed isolation level can be used to provide exactly-once delivery when reading, processing and writing data on Kafka topics.

Kafka 4.3, Design: Message Delivery Semantics

08 Store the offset with the output

If the output of processing lands in your own database, put the offset there too, in the same transaction as the result. Then there is no gap to lose work in, because there is no second commit.

Kafka explicitly supports this: offset storage is not required to live in Kafka.

This is the cleanest exactly-once you will get against an external system, and it costs you the built in offset tooling, which is a real trade rather than a free win.

Consumer applications need not use Kafka built-in offset storage; they can store offsets in a store of their own choosing. This allows storing both the offset and results of consumption in the same system atomically, which gives exactly once semantics.

Kafka 4.3, Design: Message Delivery Semantics

Managing offsets yourself means group tooling and lag monitoring no longer see your true position, so the observability has to be rebuilt around your own store.

Operational practice, not vendor documentation

09 Isolation level, and the offset that blocks

At the time of writing, September 2026, on Kafka 4.3, `isolation.level` defaults to `read_uncommitted`. A consumer left on the default will read records from transactions that were later aborted.

Set to `read_committed`, a consumer reads only up to the last stable offset, which is one less than the offset of the first open transaction.

That has a consequence worth knowing: a long running open transaction BLOCKS a `read_committed` consumer from advancing past it. The consumer is not broken and the lag is real. Somebody left a transaction open.

```
isolation.level defaults to read_uncommitted on Kafka 4.3.
```

```
Kafka 4.3, Consumer Configs
```

```
read_committed will only return messages up to the last stable offset, which is the one less than the offset of the first open transaction.
```

```
Kafka 4.3, Consumer Configs
```

```
enable.idempotence defaults to true on the producer on Kafka 4.3.
```

```
Kafka 4.3, Producer Configs
```

10 The interview answer

An offset is committed progress for a consumer group on a partition. It records where to resume, not what was done.

Commit before processing and you risk losing work. Commit after and you risk repeating it. At-least-once is the default and the honest starting point.

Exactly-once is real inside Kafka, using a transactional producer and `read_committed`. Outside Kafka it needs the other system to cooperate, and the cheapest cooperation is storing the offset alongside the output.

Then say what you would DO: make the effect idempotent, commit after, and know which isolation level you are on.

```
Answering with the trade rather than with a definition, and then naming the mitigation, is what distinguishes a senior answer from a correct one.
```

```
First principles, not vendor documentation
```

Questions to ask before you answer

1. Where does the side effect live, inside Kafka or outside it?
2. Is auto commit on, and what is the interval?
3. Is the commit before or after the effect?
4. Is the effect idempotent, and keyed on what?
5. What isolation level are the consumers on?
6. Could the offset be stored with the output instead?
7. Which systems must agree for exactly-once to mean anything here?
8. What happens to in flight work when this process is killed?

Everything else



Site

<https://code-with-sam-dev.github.io>



YouTube

<https://www.youtube.com/@CodewithSam-Dev>



LinkedIn

<https://www.linkedin.com/company/code-with-sam-dev>



X

<https://x.com/CodeWithSamDev>



TikTok

<https://www.tiktok.com/@codewithsamdev>



Instagram

<https://www.instagram.com/codewithsamdev>



Facebook

<https://www.facebook.com/codewithsamdev>



GitHub

<https://github.com/code-with-sam-dev>



Buy me a coffee

<https://buymeacoffee.com/codewithsam>



Buy me sadza

<https://buymesadza.com/codewithsam>

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which system you want taken apart next.

Apache Kafka and the Kafka logo are trademarks of the Apache Software Foundation. PostgreSQL and its logo are trademarks of the PostgreSQL Community Association. They appear here only to identify the technology being discussed. Nothing here implies endorsement by, or affiliation with, any of them.

Technical claims verified 2026-09-12 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.